

Errores y excepciones

Principios de Construcción de
Software

Manejo de excepciones

- Cuando se utilizan debidamente las excepciones mejoran la legibilidad , confiabilidad y el mantenimiento del código
- Cuando se utilizan de manera opuesta los efectos pueden ser catastróficos

Manejo de excepciones

- El propósito de un correcto manejo de excepciones está ligado a un correcto manejo de requerimientos
- Los requerimientos primarios
 - La aplicación debe sobrevivir a la excepción
 - Debe continuar procesando eventos o recursos
 - Terminar correctamente
 - Notificar a las partes interesadas: Usuarios, administradores, clientes, desarrolladores
 - El origen de la excepción debe ser reproducible para ser corregida

Manejo de excepciones

- Los requerimientos secundarios
 - Encapsulación
 - Mantenimiento
 - Definición de una buena arquitectura

Manejo de excepciones

- Es importante manejar adecuadamente las excepciones
 - Se “cacha” la excepción
 - Se maneja el error y trata de recuperarse
 - Si no es posible la recuperación, el apagado debe ser adecuado
 - Cerrar recursos adecuadamente

Manejo de excepciones

- ¿Qué estrategias de notificación de excepciones utilizarías?
- ¿A quiénes notificarías y cómo?
- ¿Qué elementos incluirías en la excepción para retroalimentar objetivamente a las partes interesadas?

Manejo de excepciones

- Tips básicos
 - Checked Exceptions: deben ser declaradas en la cláusula Throws de un método
 - Indican que se espera que un problema ocurra en una operación normal: comunicación, entradas de usuario, parámetros.
 - Unchecked: no se necesitan declarar en una cláusula Throws
 - Indican que un problema puede ser por un bug en el código
 - Resulta mejor propagarlas y mandarlas a una bitácora

Manejo de excepciones

- Crear excepciones personalizadas
 - Para enviar excepciones cuando algo va mal

```
throw new MyObjectNotFoundException("Couldn't find object id" + id);
```

- Para encapsular información extra proveniente de otra excepción

```
catch(NoSuchMethodException e){  
    throw new MyServiceException("Couldn't process request ", e);  
}
```


Manejo de excepciones

- Seguimiento (Logging)
 - Cuando se encuentra una excepción, es vital saber qué hacer con ella (manejarla, propagarla, o “loguearla”)
 - El paquete `java.util.logging` puede ser de ayuda para “loguear” excepciones
 - FATAL: casos extremos, atención inmediata
 - ERROR: Indica un bug o una condición de error
 - WARN: No es un bug, pero es algo que probablemente necesitamos saber
 - INFO: información de diagnóstico
 - DEBUG: para depuración

¿Qué mejoras harías?

```
private void abuseException(int[] range){  
    try{  
        int i=0;  
        int result=0;  
        while(true){  
            result=range[i++];  
        }  
    }catch(ArrayIndexOutOfBoundsException e){  
  
    }  
}
```

Excepciones

- Están diseñadas para manejar circunstancias excepcionales
- Poner código dentro de un bloque try-catch inhibe ciertas optimizaciones de la JVM
- Las excepciones nunca deben ser utilizadas para determinar el flujo normal del código

Excepciones

- Solo utilizar excepciones “checked” para condiciones recuperables y excepciones “runtime” para errores del programa
- Java tiene tres tipos de *throwables*
 - Checked exceptions
 - Runtime exceptions
 - Errors

Excepciones

Exception

IllegalArgumentException

IllegalStateException

NullPointerException

IndexOutOfBoundsException

ConcurrentModificationException

UnsupportedOperationException

Uso

El valor de un parámetro es inapropiado

El estado de un objeto es inapropiado para la invocación de un método

El valor nulo en un parámetro no está permitido

Índice inválido

Modificación concurrente de un parámetro es detectada y está prohibida

El objeto no soporta el método solicitado

No ignorar excepciones

//Ejemplo

```
try{
```

```
    . . .
```

```
}catch(Exception e){
```

```
}
```

Si vas a ignorar explica

- Un bloque vacío catch va en contra del propósito original para el manejo de excepciones
- Si no tienes de otra, entonces incluye un bloque que comente por qué se ignora la excepción

Incluye información de la falla

- Cuando un problema ocurre, se incluye información del problema en la pila
- El “Stack trace” incluye una representación textual de la falla
- El mensaje de falla debe contener los valores de TODOS los parámetros y campos que están incluidos en la excepción

Incluye información de la falla

```
/**
 * Construct an IndexOutOfBoundsException.
 *
 * @param lowerBound the lowest legal index value.
 * @param upperBound the highest legal index value plus one.
 * @param index      the actual index value.
 */
public IndexOutOfBoundsException(int lowerBound, int upperBound,
                                int index) {
    // Generate a detail message that captures the failure
    super("Lower bound: " + lowerBound +
          ", Upper bound: " + upperBound +
          ", Index: " + index);

    // Save failure information for programmatic access
    this.lowerBound = lowerBound;
    this.upperBound = upperBound;
    this.index = index;
}
```

Cerrar recursos

```
public void processFile(String inPath, String outPath)
    throws IOException{
    try (BufferedReader br =
        new BufferedReader(new FileReader(inPath));
        BufferedWriter bw =
            new BufferedWriter(new FileWriter(outPath));) {
        // Process the input and produce the output
    } catch (IOException ex) {
        // Print out all exceptions, including suppressed ones
        System.err.println("thrown exception: " + ex.toString());
        Throwable[] suppressed = ex.getSuppressed();
        for (int i = 0; i < suppressed.length; i++) {
            System.err.println("suppressed exception: " +
                suppressed[i].toString());
        }
    }
}
```

Cerrar recursos

```
public void processFile(String inPath, String outPath)
    throws IOException {
    BufferedReader br = null;
    BufferedWriter bw = null;
    try {
        br = new BufferedReader(new FileReader(inPath));
        bw = new BufferedWriter(new FileWriter(outPath));
        // Process the input and produce the output
    } finally {
        if (br != null) {
            try {
                br.close();
            } catch (IOException x) {
                // Handle error
            } finally {
                if (bw != null) {
                    try {
                        bw.close();
                    } catch (IOException x) {
                        // Handle error
                    }
                }
            }
        }
    }
}
```

Antipatrones

Log and Throw

Example:

```
catch (NoSuchMethodException e) { LOG.error("Blah", e); throw e; }
```

or

```
catch (NoSuchMethodException e) { LOG.error("Blah", e); throw new  
MyServiceException("Blah", e); }
```

or

```
catch (NoSuchMethodException e) { e.printStackTrace(); throw new  
MyServiceException("Blah", e); }
```

All of the above examples are equally wrong. This is one of the most annoying error-handling antipatterns. Either log the exception, or throw it, but never do both. Logging and throwing results in multiple log messages for a single problem in the code, and makes life hell for the support engineer who is trying to dig through the logs.

Antipatrones

Throwing Exception

Example:

```
public void foo() throws Exception {
```

This is just sloppy, and it completely defeats the purpose of using a checked exception. It tells your callers "something can go wrong in my method." Real useful. Don't do this. Declare the specific checked exceptions that your method can throw. If there are several, you should probably wrap them in your own exception

Antipatrones

Throwing the Kitchen Sink

Example:

public void foo() throws MyException, AnotherException, SomeOtherException, YetAnotherException

Throwing multiple checked exceptions from your method is fine, as long as there are different possible courses of action that the caller may want to take, depending on which exception was thrown. If you have multiple checked exceptions that basically mean the same thing to the caller, wrap them in a single checked exception.

Antipatrones

Catching Exception

Example:

```
try { foo(); } catch (Exception e) { LOG.error("Foo failed", e); }
```

This is generally wrong and sloppy. Catch the specific exceptions that can be thrown. The problem with catchingException is that if the method you are calling later adds a new checked exception to its method signature, the developer's intent is that you should handle the specific new exception. If your code just catches Exception (or worse, Throwable), you'll probably never know about the change and the fact that your code is now wrong.

Antipatrones

Destructive Wrapping

Example:

```
catch (NoSuchMethodException e) { throw new MyServiceException("Blah: " +  
e.getMessage()); }
```

This destroys the stack trace of the original exception, and is always wrong.

Log and Return Null

Example:

```
catch (NoSuchMethodException e) { LOG.error("Blah", e); return null; }
```

or

```
catch (NoSuchMethodException e) { e.printStackTrace(); return null; } // Man I hate this one
```

Although not always incorrect, this is usually wrong. Instead of returning null, throw the exception, and let the caller deal with it. You should only return null in a normal (non-exceptional) use case (e.g., "This method returns null if the search string was not found.").

Antipatrones

Catch and Ignore

Example:

```
catch (NoSuchMethodException e) { return null; }
```

This one is insidious. Not only does it return null instead of handling or re-throwing the exception, it totally swallows the exception, losing the information forever.

Throw from Within Finally

Example:

```
try { blah(); } finally { cleanUp(); }
```

This is fine, as long as `cleanUp()` can never throw an exception. In the above example, if `blah()` throws an exception, and then in the finally block, `cleanUp()` throws an exception, that second exception will be thrown and the first exception will be lost forever. If the code that you call in a finally block can possibly throw an exception, make sure that you either handle it, or log it. Never let it bubble out of the finally block.

Antipatrones

Multi-Line Log Messages

Example:

```
LOG.debug("Using cache policy A"); LOG.debug("Using retry policy B");
```

Always try to group together all log messages, regardless of the level, into as few calls as possible. So in the example above, the correct code would look like:

```
LOG.debug("Using cache policy A, using retry policy B");
```

Using a multi-line log message with multiple calls to `log.debug()` may look fine in your test case, but when it shows up in the log file of an app server with 500 threads running in parallel, all spewing information to the same log file, your two log messages may end up spaced out 1000 lines apart in the log file, even though they occur on subsequent lines in your code.

Unsupported Operation Returning Null

Example:

```
public String foo() { // Not supported in this implementation. return null; }
```

When you're implementing an abstract base class, and you're just providing hooks for subclasses to optionally override, this is fine. However, if this is not the case, you should throw an `UnsupportedOperationException` instead of returning null. This makes it much more obvious to the caller why things aren't working, instead of her having to figure out why her code is throwing some random `NullPointerException`.

Antipatrones

IgnoringInterruptedException

Example:

```
while (true) { try { Thread.sleep(100000); } catch (InterruptedException e) {}  
doSomethingCool(); }
```

InterruptedException is a clue to your code that it should stop whatever it's doing. Some common use cases for a thread getting interrupted are the active transaction timing out, or a thread pool getting shut down. Instead of ignoring the InterruptedException, your code should do its best to finish up what it's doing, and finish the current thread of execution. So to correct the example above:

```
while (true) { try { Thread.sleep(100000); } catch (InterruptedException e) { break; }  
doSomethingCool(); }
```

Antipatrones

Relying on `getCause()`

Example:

```
catch (MyException e) { if (e.getCause() instanceof FooException) { ...
```

The problem with relying on the result of `getCause()` is that it makes your code fragile. It may work fine today, but what happens when the code that you're calling into, or the code that it relies on, changes its underlying implementation, and ends up wrapping the ultimate cause inside of another exception? Now calling `getCause()` may return you a wrapping exception, and what you really want is the result of `getCause().getCause()`. Instead, you should unwrap the causes until you find the ultimate cause of the problem. Apache's commons-lang project provides `ExceptionUtils.getRootCause()` to do this easily.

Mejores prácticas

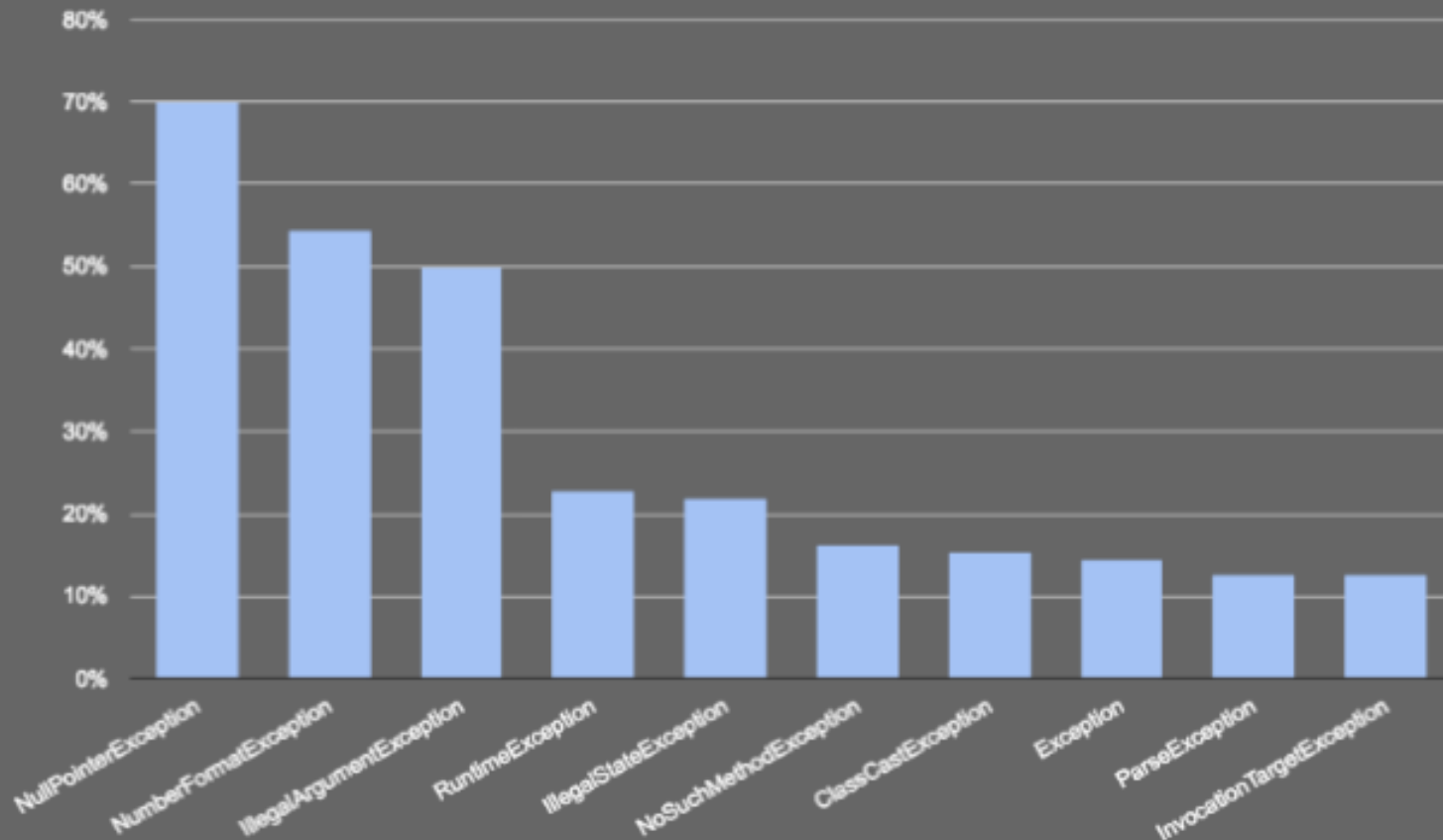
- Usar excepciones solo en casos excepcionales
- Utilizar checked exceptions para recuperación y runtime exceptions para errores de programación
- Evita el uso excesivo e innecesario de checked exceptions
- Prefiere el uso de excepciones estándar
- Utiliza niveles de abstracción adecuados

Mejores prácticas

- Documenta todas las excepciones lanzadas por un método
- Incluye información de la falla en mensajes detallados
- No ignores excepciones

Datos

Top 10 Exception Types by Frequency in 1,000+ Applications



Actividad: Excepciones

- Objetivo: que el estudiante comprenda el uso de excepciones y los tipos existentes
- Descripción: Realizar la lectura del material de apoyo y contestar las siguientes preguntas de reflexión:
 - ¿Qué diferencia hay entre error y excepción?
 - ¿Cuáles son los diferentes tipos de excepciones y cuál es el objetivo de cada una de ellas?
 - ¿A qué se refiere el término “Propagación de excepciones”?

Práctica

- Escribir una calculadora de Strings para operaciones básicas suma, resta, multiplicación y división
- Casos
 - “4+25” = 29
 - “20/0” = Excepción con mensaje “División por cero no está soportada”
 - “5/1”=5
 - Null o vacío = Excepción con mensaje “Parámetro de entrada incorrecto”
 - “5+30”=150
 - “5%6”= Excepción con mensaje “Operación no soportada”

Práctica

- Utilice las siguientes referencias de apoyo
 - <https://docs.oracle.com/javase/8/docs/api/java/lang/package-tree.html>
 - <https://docs.oracle.com/javase/8/docs/api/java/lang/Throwable.html>
 - <https://docs.oracle.com/javase/8/docs/api/java/lang/Error.html>